AI-BASED DIGITAL EMPLOYEE USING AN AGENT

A Report for the subject of Software Engineering Lab

Bachelor of Technology in
Computer Science and Engineering

Submitted by:
Name: Samuel Priyatam Dash
Registration no.: 2302040025
Semester: 6th
Subject name: Software Engineering (SE) Lab

Under the esteemed guidance of
Prof. Himansu Sekhar Behera
Professor and Head

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VEER SURENDRA SAI UNIVERSITY OF TECHNOLOGY, BURLA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VEER SURENDRA SAI UNIVERSITY OF TECHNOLOGY, BURLA

CERTIFICATE

This is to certify that the project entitled "AI-based Digital Employee using an Agent" is being submitted by Mr. Samuel Priyatam Dash (Regd. No. 2302040025) from the Department of Computer Science and Engineering, Veer Surendra Sai University of Technology, Burla. This report is an original work carried out by him under our supervision and guidance.

Date: 13/04/2026

Prof. Himansu Sekhar Behera
Head of Department, Supervisor
Computer Science and Engineering
VSSUT, Burla

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VEER SURENDRA SAI UNIVERSITY OF TECHNOLOGY, BURLA

CERTIFICATE FOR APPROVAL

This is to certify that the project report entitled "AI-based Digital Employee using an Agent" is being submitted by Mr. Samuel Priyatam Dash (Regd. No. 2302040025), a student of the Department of Computer Science and Engineering, Veer Surendra Sai University of Technology, Burla, has been thoroughly examined and evaluated by us. This report is an original work carried out by the candidate under our supervision and guidance, and it meets the requirements for submission as per the academic guidelines of the university.

Ms. Swarupa Panda
Examiner, Guest Faculty
Computer Science and Engineering
VSSUT, Burla

Prof. Himansu Sekhar Behera
Head of Department, Supervisor
Computer Science and Engineering
VSSUT, Burla

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VEER SURENDRA SAI UNIVERSITY OF TECHNOLOGY, BURLA

Declaration

I hereby declare that the project work entitled "AI-based Digital Employee using an Agent" is the result of my original research and effort, carried out under the esteemed supervision of Prof. Himansu Sekhar Behera, Department of Computer Science and Engineering, Veer Surendra Sai University of Technology, Burla. I confirm that this project is an authentic piece of work and has not been submitted, either in part or in full, for the award of any degree, diploma, or certification at any other institution or university. All sources of information used in this report have been duly acknowledged.

`

Samuel Priyatam Dash
Regd No. - 2302040025

Acknowledgement

I express my sincere gratitude to my supervisor, Prof. Himansu Sekhar Behera, for providing me with the opportunity to undertake this project. I am deeply appreciative of his insightful guidance, timely advice, continuous support, and motivation throughout my research. His kindness and invaluable contributions have been instrumental in the successful completion of this work.

I extend my heartfelt thanks to all the faculty and staff members of the Department of Computer Science and Engineering, Veer Surendra Sai University of Technology, Burla, India, for their constant support and assistance. I am especially grateful to Prof. Dipak Kumar Sahoo, Vice Chancellor, VSSUT, Burla, and Prof. Himansu Sekhar Behera, Head of the Department, Department of Computer Science and Engineering, for their encouragement and for providing me with the necessary resources and facilities throughout my academic journey.

Finally, I would like to express my deepest gratitude to my family for their unwavering love, encouragement, and support, which have been my greatest source of strength.

`

Samuel Priyatam Dash
Regd No. - 2302040025

Abstract

The rapid advancement of artificial intelligence and automation technologies has transformed the way organizations manage operations and decision-making processes. In modern enterprises, a significant amount of managerial effort is spent on repetitive, rule-based, and multi-step tasks that require coordination across different systems and platforms. This project proposes the design and conceptual development of an **AI-Based Digital Employee using an Agentic Architecture**, aimed at automating such tasks efficiently.

The system introduces a digital employee in the form of an intelligent software agent that can receive high-level instructions from managers and autonomously execute them. The agent interprets instructions, decomposes them into structured tasks, and performs actions by interacting with external tools and services through APIs and integrations. The system ensures seamless execution by incorporating task management, decision-making, and logging mechanisms.

To systematically model the system, various Software Engineering methodologies have been applied. The **Entity Relationship (ER) model** defines the core data structure, including entities such as users, requests, tasks, decisions, and system logs. The **Data Flow Diagrams (DFD)** illustrate how data moves through different levels of the system, from user input to execution and output generation. Additionally, **Use Case Diagrams** and **Structure Charts** provide a clear understanding of system functionality and modular design.

The proposed system emphasizes scalability, modularity, and automation. By reducing manual intervention, improving efficiency, and enabling intelligent decision support, the AI-based digital employee can significantly enhance productivity in organizational workflows. This project lays a strong foundation for future development of fully autonomous enterprise agents capable of handling complex business processes.

TABLE OF CONTENTS

1. PROBLEM STATEMENT & LIST OF ACTORS 8

- 1.1 Objective 8
- 1.2 Problem Description 8
- 1.3 List of Actors 9
- 1.4 Conclusion 9

2. SOFTWARE REQUIREMENT SPECIFICATION (SRS) 10

- 2.1 Introduction 10
- 2.2 Overall Description 11
- 2.3 External Interface Requirements 13
- 2.4 Functional Requirements Analysis 14
- 2.5 Non-Functional Requirements 17
- 2.6 System Constraints 18
- 2.7 Conclusion 18

3. ER MODEL (ENTITY RELATIONSHIP MODEL) 19

- 3.1 Objective 19
- 3.2 Entities and Attributes 19
- 3.3 Entity Relationship Mapping 21
- 3.4 Entity Relationship Diagram 22

4. DATA FLOW DIAGRAM (DFD) 23

- 4.1 DFD Level 0 (Context Diagram) 23
- 4.2 DFD Level 1 24
- 4.3 DFD Level 2 (Detailed Breakdown) 24
- 4.4 Structure Chart 25

5. UML DIAGRAMS 26

- 5.1 Use Case Diagram 26
- 5.2 Class Diagram 27
- 5.3 Object Diagram 28
- 5.4 Activity Diagram 29
- 5.5 State Chart Diagram 30
- 5.6 Sequence Diagram 31
- 5.7 Collaboration Diagram 32

6. CONCLUSION 33

1. Problem Statement & List of Actors

1.1 Objective

To prepare the problem statement for any project and find the list of end users using the system.

1.2 Problem Description

In many organizations, managers and team leads regularly assign routine tasks such as data collection, report generation, information retrieval, updating records, and coordinating actions across multiple software tools. These tasks are often repetitive, time-consuming, and require continuous human involvement, even though they do not demand complex decision-making.

With the increasing use of digital tools and online services, human employees are required to manually switch between different applications to complete assigned work. This leads to inefficiency, delays, and increased operational overhead. Managers also need to closely monitor task execution and follow up frequently, which further increases workload.

To address this issue, there is a need for a digital employee that can receive instructions directly from a manager and execute assigned tasks automatically. Such a digital employee should be capable of understanding task instructions, selecting appropriate actions, interacting with existing software tools, and reporting task status back to the manager.

The proposed system focuses on designing an **AI-based digital employee using an agent approach**, where the digital employee acts as an independent worker that assists human employees rather than replacing them. The digital employee performs routine tasks autonomously while allowing human supervision and intervention when required.

The objective of this project is to reduce manual effort, improve efficiency, and support managers by delegating repetitive operational tasks to a digital employee that operates based on given instructions.

1.3 List of Actors

1. **Manager / Supervisor**
- Provides task instructions to the digital employee.
- Reviews task execution results and progress reports.
- Intervenes when tasks require human decision-making.

2. **Human Employee**
- Collaborates with the digital employee.
- Handles tasks escalated by the digital employee.
- Uses outputs generated by the digital employee.

3. **Digital Employee (AI Agent)**
- Receives instructions from the manager.
- Interprets tasks and determines required actions.
- Executes tasks using available tools.
- Reports task completion status or errors.

4. **System Administrator**
- Configures the digital employee.
- Manages access permissions and system settings.
- Monitors system performance and logs.

5. **External Tools / Services**
- Software applications or platforms used by the digital employee to complete tasks.
- May include data sources, communication tools, or task execution utilities.

1.4 Conclusion

The problem statement for the AI-Based Digital Employee Using an Agent Approach was successfully formulated from an organizational perspective without describing implementation details. Based on the problem description, the key actors interacting with the system were identified and documented. This problem statement and actor list will serve as the foundation for requirement analysis, system modeling, and preparation of the Software Requirement Specification (SRS) document.

2. Software Requirements Specification (SRS) Document for AI-Based Digital Employee Using an Agent

2.1. Introduction

2.1.1 Purpose

The AI-Based Digital Employee Using an Agent is designed to automate routine, repetitive, and multi-step business processes that currently require manual coordination across different software tools. The system aims to provide a centralized platform where managers can delegate complex tasks to an autonomous software worker capable of reasoning, planning, and executing actions through various APIs. The system automates manual coordination tasks, ensuring consistency in record updates, report generation, and data retrieval to improve organizational productivity and support managers in high-level decision-making.

2.1.2 Document Conventions

This document follows standard SRS formatting guidelines:

- Bold headings for major sections.
- Numbered sections for easy navigation.
- Bullet points to present key details clearly.

2.1.3 Intended Audience and Reading Suggestions

This document is intended for various stakeholders involved in enterprise automation:

- Managers and Supervisors – Oversee the delegation of tasks, review final outcome reports, and provide manual approvals for gated actions.
- Human Employees – Collaborate with the digital employee by providing domain context or handling tasks escalated due to high emotional intelligence requirements.
- IT Professionals and Developers – Implement the underlying reasoning engine, integrate external tool APIs, and manage the Model Context Protocol gateway.
- System Administrators – Configure system guardrails, manage user access permissions, and monitor activity and compliance logs.

2.1.4 Product Scope

The AI-Based Digital Employee aims to provide a comprehensive autonomous solution for organizational operations, reducing manual bottlenecks and human error. The system will replace fragmented manual workflows with an intelligent, agent-based platform that enables:

- Natural language interpretation of high-level managerial instructions.
- Autonomous task planning and hierarchical decomposition of complex goals.
- Secure tool actuation through standardized API integrations and SaaS connectors.
- Stateful session persistence to maintain context over long-running workflows.
- Immutable activity logging and human-in-the-loop approval gating for risk management.

  The system will be an enterprise-grade runtime with a web-based dashboard for desktop and mobile devices, integrating secure authentication, encrypted data movement, and a scalable memory architecture for handling multiple concurrent agent sessions and diverse business processes.

2.2. Overall Description

2.2.1 Product Perspective

The AI-Based Digital Employee is a standalone software application designed to act as an intelligent intermediary between human intent and existing enterprise infrastructure. It offers seamless integration with CRM systems, ERP platforms, communication tools, and internal databases through standard REST APIs and the Model Context Protocol. It provides a centralized reasoning hub where stakeholders can manage autonomous workflows in real time without the need for manual script-based automation.

2.2.2 Product Functions

The system will include the following key functions:

- **Instruction Ingestion and Parsing**: Managers can provide instructions via text or voice which the system analyzes to initialize a goal-oriented session.
- **Strategic Planning and Reasoning**: The system breaks down abstract goals into a logical sequence of subtasks using advanced reasoning loops.
- **Knowledge Augmentation**: The agent retrieves relevant organizational data from a vector store to ground its decisions in current policy.
- **Tool Discovery and Actuation**: The system identifies the most appropriate authorized API to perform actions such as sending emails or updating database records.
- **Progress Monitoring and Status Tracking**: Users can track the agent's internal reasoning steps and the completion status of each subtask in real time.
- **Self-Correction and Error Recovery**: If a tool fails or an observation is unexpected, the agent autonomously revises its plan to find an alternative path.
- **Approval Gating**: High-consequence actions are paused by the system for explicit manual authorization by a supervisor.
- **Outcome Synthesis and Reporting**: The system generates detailed summaries of task results, highlighting deviations from the original plan.
- **Governance and Scope Enforcement**: The system restricts agent actions to a predefined "Agency Perimeter" to ensure organizational compliance.

2.2.3 User Classes and Characteristics

- Managers and Supervisors: Assign goals and review reports; require minimal technical expertise but high domain authority.
- Human Employees: Collaborate on complex tasks; handle escalations; require medium technical knowledge and high process awareness.
- System Administrators: Configure security guardrails and manage model deployments; require high technical expertise and administrative rights.
- IT Developers and AI Engineers: Integrate new tools and optimize reasoning latency; require very high technical expertise in APIs and machine learning.

2.2.4 Operating Environment

- Platform: Linux-based backend system (Ubuntu 22.04+) optimized for containerized deployments.
- Model Execution: Support for cloud-managed models (e.g., GPT-4o) and local inference engines (e.g., vLLM or Ollama).
- Database: PostgreSQL for persistent task storage and activity logs; Redis for sub-millisecond session state management.
- Security: Zero-trust architecture with AES-256 encryption at rest and TLS 1.3 encryption in transit.

2.2.5 Design and Implementation Constraints

- Regulatory Compliance: Must adhere to data protection laws such as GDPR Article 22 regarding automated individual decision-making.
- Hardware Limitations: Local inference for large models (70B+ parameters) requires high memory bandwidth and significant VRAM.
- Deterministic Contracts: All tool interactions must strictly follow JSON Schema or OpenAPI standards to prevent reasoning hallucinations.
- Safety Guardrails: The system must operate within an agentic harness that prevents unauthorized tool use regardless of model instructions.

2.2.6 User Documentation

- Manager Onboarding Guide for effective instruction briefing.
- Administrator Setup and Security Manual for configuring guardrails.
- API and Tool Integration Kit for technical developers.
- Governance Policy Handbook for organizational compliance officers.

2.2.7 Assumptions and Dependencies

- Managers provide instructions with enough clarity to identify a successful end-state.
- The system assumes stable internet connectivity for accessing cloud-based models and third-party SaaS APIs.
- Successful tool execution depends on the availability and responsiveness of integrated enterprise software.

2.3. External Interface Requirements

2.3.1 User Interface

The system will feature an intuitive and transparent user interface designed for diverse organizational roles.

Key UI Features:

- Instruction Chat Interface – Real-time terminal for natural language task briefing.
- Workflow Graph Visualizer – Dynamic node-based view showing the decomposition of goals into active subtasks.
- Approval Portal – Dedicated dashboard for managers to review gated actions and agent rationales.
- Governance and Policy Dashboard – Panel for defining permission boundaries and reviewing audit logs.
- Performance Analytics – Reports on task completion rates, token usage, and response latency.

2.3.2 Hardware Interface

The system requires specific computing hardware depending on the deployment tier.

Server-Side (Local Inference):

- Device: High-performance workstation or rack-mounted server.
- Processor: Minimum 16-core CPU for orchestration and preprocessing.
- RAM: 128GB DDR5 ECC or higher.
- GPU: NVIDIA RTX 4090 or A100 with 24GB to 80GB VRAM. Client-Side (User Access):
- Device: Any modern smartphone, tablet, or personal computer.
- Internet Connectivity: Required for accessing the web-based dashboard and cloud APIs.

2.3.3 Software Interface

The system integrates with multiple third-party software components and services.

- Operating Systems: Linux (Ubuntu 22.04+) for servers; cross-platform support for mobile and desktop browsers.
- Inference APIs: Connectors for cloud LLM providers including OpenAI, Anthropic, and Google Vertex AI.
- Model Context Protocol (MCP): Standardized gateway for dynamic tool discovery and communication.
- Identity Brokers: Integration with Okta or Auth0 for role-based access control and secure token management.
- Enterprise Connectors: Bi-directional integrations with Salesforce, NetSuite, Slack, Workday, and Jira.

2.3.4 Communication Interface

The system supports several communication channels for internal and external interaction.

1. Internal Communication:
- RESTful APIs to connect the frontend UI with the reasoning backend.
- WebSockets for streaming real-time status updates and "thoughts" to the user.
2. External Communication:
- SMTP/Secure Email: For sending synthesized task reports and error notifications.
- Webhooks: For receiving events and triggers from integrated SaaS platforms.
3. Network Communication:
- Deployment on cloud providers (AWS, Azure, or GCP) for high availability.
- HTTPS Protocol (TLS 1.3) for securing all data transmission.

2.4 Functional Requirements for AI-Based Digital Employee

R1. Manager Instruction Ingestion

- Description: Allows authorized managers to provide high-level goals via text or voice to initialize the session.
- Input: Natural language instruction, manager authentication token.
- Output: Unique Session ID, receipt confirmation.
- Processing: The system validates identity, sanitizes input for prompt injection, and initializes the reasoning loop.

R2. Intent Analysis and Mapping

- Description: Analyzes instructions to identify the core objective and ensures it falls within the agent's mandate.
- Input: Raw instruction text.
- Output: Structured intent JSON containing goal entities.
- Processing: Uses LLM perception layers to parse intent; requests clarification if the goal is outside permitted scopes.

R3. Strategic Task Planning

- Description: Formulates a sequence of steps to achieve the identified goal, considering priorities and constraints.
- Input: Structured intent JSON.
- Output: Hierarchical Task Network (HTN) or execution graph.
- Processing: The system uses Chain-of-Thought reasoning to visualize dependencies and optimal execution order.

R4. Dynamic Task Decomposition

- Description: Breaks strategic plan steps into discrete, executable tool calls or sub-activities.
- Input: Strategic plan graph.
- Output: List of primitive subtasks with associated parameters.
- Processing: Maps abstract objectives (e.g., "Verify status") to specific software functions (e.g., "Query Database").

R5. Contextual Knowledge Retrieval (RAG)

- Description: Retrieves organizational context for each reasoning step to ensure alignment with company policy.
- Input: Reasoning context, query embedding.
- Output: Relevant document chunks and policy constraints.
- Processing: Performs semantic search against a vector database and injects context into the current LLM reasoning window.

R6. Tool Discovery and Selection

- Description: Identifies the most appropriate authorized API or script to execute a specific primitive action.
- Input: Subtask description.
- Output: Tool ID, API endpoint, and required permission scope.
- Processing: Queries the Tool Manifest via the MCP Gateway for semantic matches between task requirements and tool capabilities.

R7. Secure Tool Actuation

- Description: Invokes the selected tool while managing authentication, parameter validation, and data movement.
- Input: Tool ID, task parameters (JSON).
- Output: Execution result or response observation.
- Processing: Validates inputs against JSON Schemas and executes calls via a secure vault for credentials.

R8. Stateful Session Persistence

- Description: Maintains context across long-running or multi-turn business workflows to ensure continuity.
- Input: Interaction history, task observations.
- Output: Updated and persisted session state.
- Processing: Stores immediate state in Redis and periodic checkpoints in a transactional database to allow recovery.

R9. Autonomous Self-Correction

- Description: Detects failures and autonomously attempts alternative execution paths or retry sequences.
- Input: Error codes, negative observations.
- Output: Revised plan or retry log.
- Processing: Uses reflective reasoning to analyze failures and adjust parameters for subsequent attempts.

R10. Human-in-the-Loop Approval Gating

- Description: Pauses execution for manual manager review before performing high-risk or sensitive actions.
- Input: Gated action proposal, risk threshold.
- Output: Approval status, manager feedback.
- Processing: Intercepts calls to sensitive APIs and presents the rationale and proposed data payload to the manager dashboard.

R11. Multi-Agent Orchestration

- Description: Coordinates specialized sub-agents for tasks spanning multiple domains or departments.
- Input: Complex goal requiring diverse expertise.
- Output: Coordinated agent outputs.
- Processing: A supervisor agent allocates sub-goals to specialized workers via an event-driven message bus.

R12. Activity Logging and Auditing

- Description: Maintains an immutable, timestamped record of every internal decision and external interaction.
- Input: System events, reasoning traces.
- Output: Audit logs and compliance reports.
- Processing: Tags all events with a correlation ID and stores them in an encrypted format suitable for forensic review.

R13. Outcome Synthesis and Reporting

- Description: Synthesizes final results and reasoning logs into a human-readable summary report for the manager.
- Input: Execution history, final observations.
- Output: Synthesized Report (PDF/Markdown).
- Processing: Uses an LLM to summarize the journey from instruction to outcome, highlighting key findings.

R14. Governance and Permission Management

- Description: Enables administrators to define the boundaries of autonomy for each digital employee session.
- Input: Policy definitions, RBAC matrices.
- Output: Applied security constraints.
- Processing: Enforces least-privilege boundaries at the infrastructure level to prevent unauthorized tool access.

R15. Knowledge Base Synchronization

- Description: Periodically updates the internal vector store with the latest organizational documentation.
- Input: New document uploads, task logs.
- Output: Updated vector embeddings.
- Processing: An automated indexing pipeline parses new data and refreshes the semantic memory store for future reasoning.

2.5 Non-Functional Requirements

Performance Requirements

- The system shall initiate response generation within 200 milliseconds in 98% of cases.
- The system shall support up to 100 concurrent active agent sessions per node with less than 1% performance degradation.

Scalability Requirements

- The system must be able to scale horizontally to handle 10-fold spikes in task utilization during peak organizational periods.
- The architecture should support the modular registration of new external tools without requiring a system reboot.

Security Requirements

- All sensitive credentials and PII must be encrypted at rest using AES-256 and in transit via TLS 1.3.
- The system shall implement prompt injection detection capable of blocking 99% of adversarial jailbreak attempts.

Usability Requirements

- The instruction interface must be intuitive, requiring minimal training for non-technical managers.
- The system should provide a clear "reasoning trace" to explain decisions and maintain user trust.

Reliability Requirements

- The core orchestrator shall maintain a 99.99% uptime availability.
- The system must persist state snapshots to allow instant recovery of workflows in case of server failure.

2.6 System Constraints

- **Device and Platform Dependency**

  The system should run on Linux-based servers optimized for GPU acceleration and high memory bandwidth. The user interface must be compatible with standard web browsers such as Chrome, Firefox, and Safari across desktop and mobile devices.

- **Internet Connectivity**

  Stable high-speed network connectivity (minimum 1 Gbps) is required for real-time interaction with cloud models and external SaaS APIs. Offline functionality is restricted to tasks involving pre-indexed local data.

- **Data Privacy**

  The system must comply with data privacy regulations such as GDPR and the EU AI Act. Personal information must be handled using data minimization principles and stored securely within regional residency boundaries.

- **Resource Limitations**

  System performance for local reasoning is constrained by VRAM capacity. Local inference for 70B+ parameter models requires a minimum of 48GB VRAM to achieve responsive token generation speeds.

2.7 Conclusion

The requirements for the AI-Based Digital Employee have been successfully documented to align with organizational efficiency and governance standards. By focusing on goal-oriented reasoning, secure tool integration, stateful memory, and transparent activity reporting, the system will provide a resilient platform for automating complex business workflows. The system is designed to handle current operational needs while offering future scalability, ensuring a productive and secure environment for collaboration between human staff and their digital counterparts.

3. Entity Relationship Diagram

3.1 Objective

Understanding the Entity Relation Model For AI-Based Digital Employee
Figure: Entity Relationship Diagram

3.2 Entities and Their Descriptions

3.2.1. User

The User entity represents all individuals interacting with the system, including managers, human employees, and system administrators.

1. Name: The full name of the user.
2. Email: The email address used for login and notifications.
3. Role: Defines the user's role in the system (Manager, Human Employee, Administrator).
4. Department: The organizational department the user belongs to.
5. Password_Hash: Secure credential for authentication.
6. Access_Level: Permissions assigned to the user for task assignment or configuration.

3.2.2. AI Agent

The AI Agent entity represents the "Digital Employee" instances created to perform work.

- Agent_ID: A unique identifier for the digital employee instance.
- Handle: A unique, machine-addressable identifier used for logging and routing.
- Model_Type: The reasoning engine powering the agent (e.g., Llama 3, GPT-4o).
- Capabilities: A list of tasks the agent is authorized to perform.
- Status: Current operational state (Idle, Reasoning, Executing, Paused).

3.2.3. Instruction

The Instruction entity represents the high-level goals provided by a manager.

- Instruction_ID: A unique identifier for the manager's request.
- Manager_ID: A foreign key referencing the User table, linking the goal to its assigner.
- Raw_Briefing: The natural language text or voice transcript provided.
- Timestamp: The date and time when the instruction was received.
- Priority: The urgency level of the request (High, Medium, Low).

3.2.4. Task

The Task entity represents a discrete, executable sub-unit of a parent instruction.

- Task_ID: A unique identifier for each decomposed subtask.
- Instruction_ID: A foreign key referencing the Instruction table.
- Task_Name: Human-readable name for the action (e.g., "Verify Invoice Amount").
- Status: Completion status (Pending, In Progress, Completed, Failed).
- Retry_Limit: Maximum number of autonomous recovery attempts.
- Dependency_ID: Foreign key linking to another task that must be completed first.

3.2.5. Tool

The Tool entity represents the standardized interface to an external system or API.

- Tool_ID: A unique identifier for the API or script.
- Tool_Name: The name of the software or service (e.g., "Salesforce API").
- API_Endpoint: The target URL for the tool actuation.
- Interface_Contract: The JSON Schema or spec governing inputs/outputs.
- Permission_Scope: The specific authorized actions for this tool instance.

3.2.6. Execution Log

The Execution Log entity records the granular Think-Act-Observe cycles of the agent.

- Log_ID: A unique identifier for each reasoning step.
- Task_ID: A foreign key referencing the Task table.
- Agent_Thought: The reasoning string generated by the LLM before acting.
- Action_Command: The exact tool call parameters sent to an API.
- Observation: The raw data or result returned by the tool.
- Execution_Time: Timestamp of the step.

3.2.7. Knowledge Base (Memory)

The Knowledge Base entity represents chunks of organizational data used for RAG.

- MemoryID: A unique lookup identifier for the information chunk.
- Knowledge_Source: Reference to the original document or policy file.
- Text_Payload: The human-readable content used to ground reasoning.
- Vector_Embedding: The mathematical representation for semantic search.

3.2.8. Approval Request

The Approval Request entity handles human-in-the-loop gating for high-risk tasks.

- ApprovalID: A unique identifier for the manual gating instance.
- Task_ID: A foreign key linking the request to a specific subtask.
- Approver_ID: A foreign key referencing the User (Manager) responsible.
- Rationale: The agent's explanation for why it needs approval.
- Decision: The manager's response (Approve, Reject, Modify).

3.3 Relationships and Their Descriptions

3.3.1. One-to-Many Relationships

- A User can assign multiple Instructions: A single manager can provide numerous high-level goals, but each instruction is owned by one manager.
- An Instruction can generate multiple Tasks: Decomposing a complex goal results in several subtasks, all belonging to the parent instruction.
- A Task can produce multiple Execution Logs: A single task may require several iterative reasoning loops (Think-Act-Observe) to complete.
- An AI Agent can execute multiple Tasks: A digital employee instance can handle various sub-activities across different instructions.
- A Manager can review multiple Approval Requests: A supervisor oversees all gated actions triggered by various tasks.
- An Instruction can trigger multiple AI Agent sessions: Complex workflows may spawn several specialized agents.

3.3.2. Many-to-Many Relationships (Handled via Mapping Tables)

- Tasks & Tools (Handled via Tool Execution Mapping): A task may require multiple tools to complete, and a single tool (like a database query) can be used by multiple tasks.
- Tasks & Knowledge Base (Handled via Retrieval Mapping): The agent retrieves multiple snippets from the knowledge base to ground a single task, and one policy snippet may inform many different tasks.

3.4 Entity Relationship Diagram

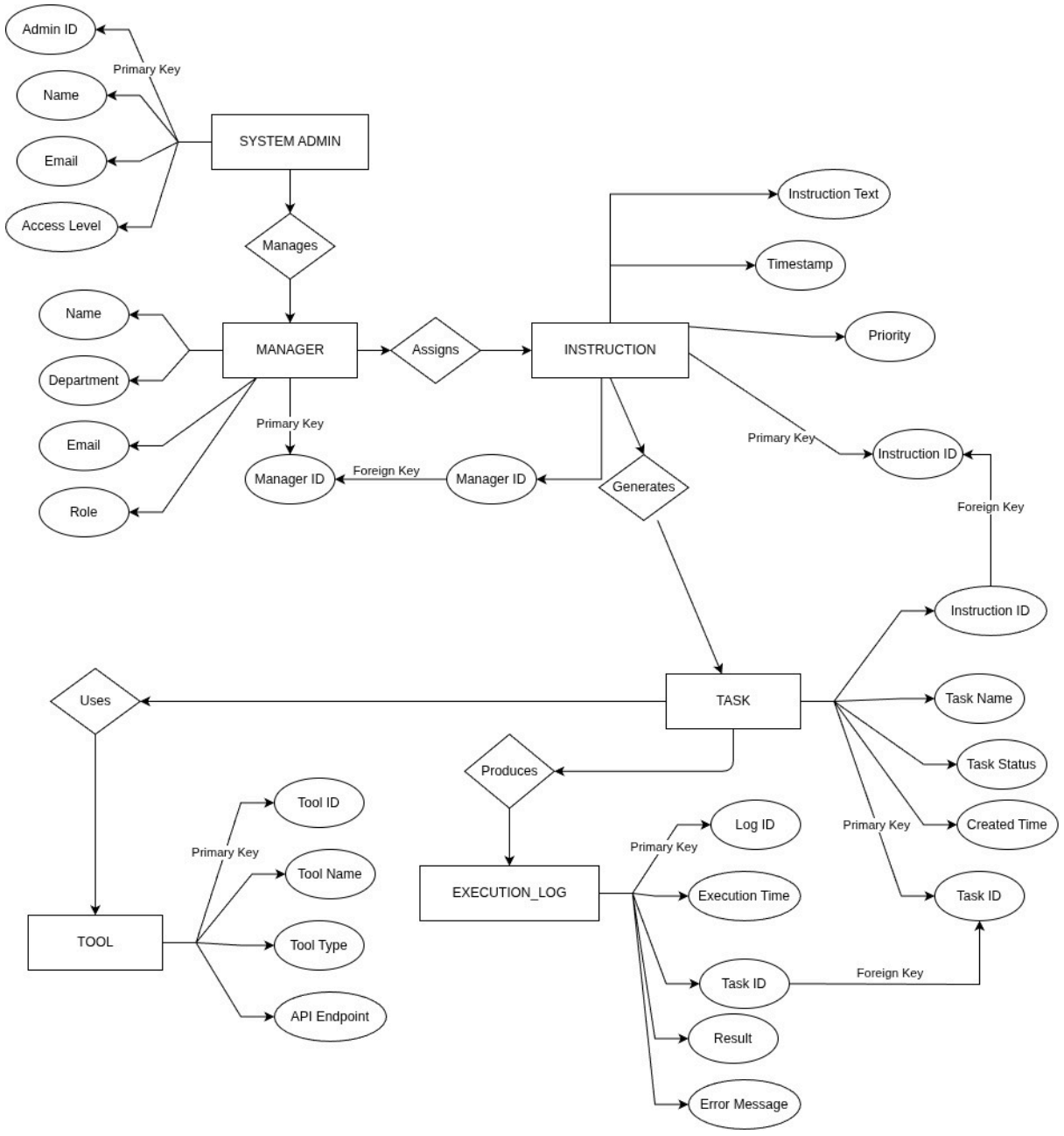

Figure 1

4. DATA FLOW DIAGRAM (DFD)

4.1 DFD Level 0 (Context Diagram)

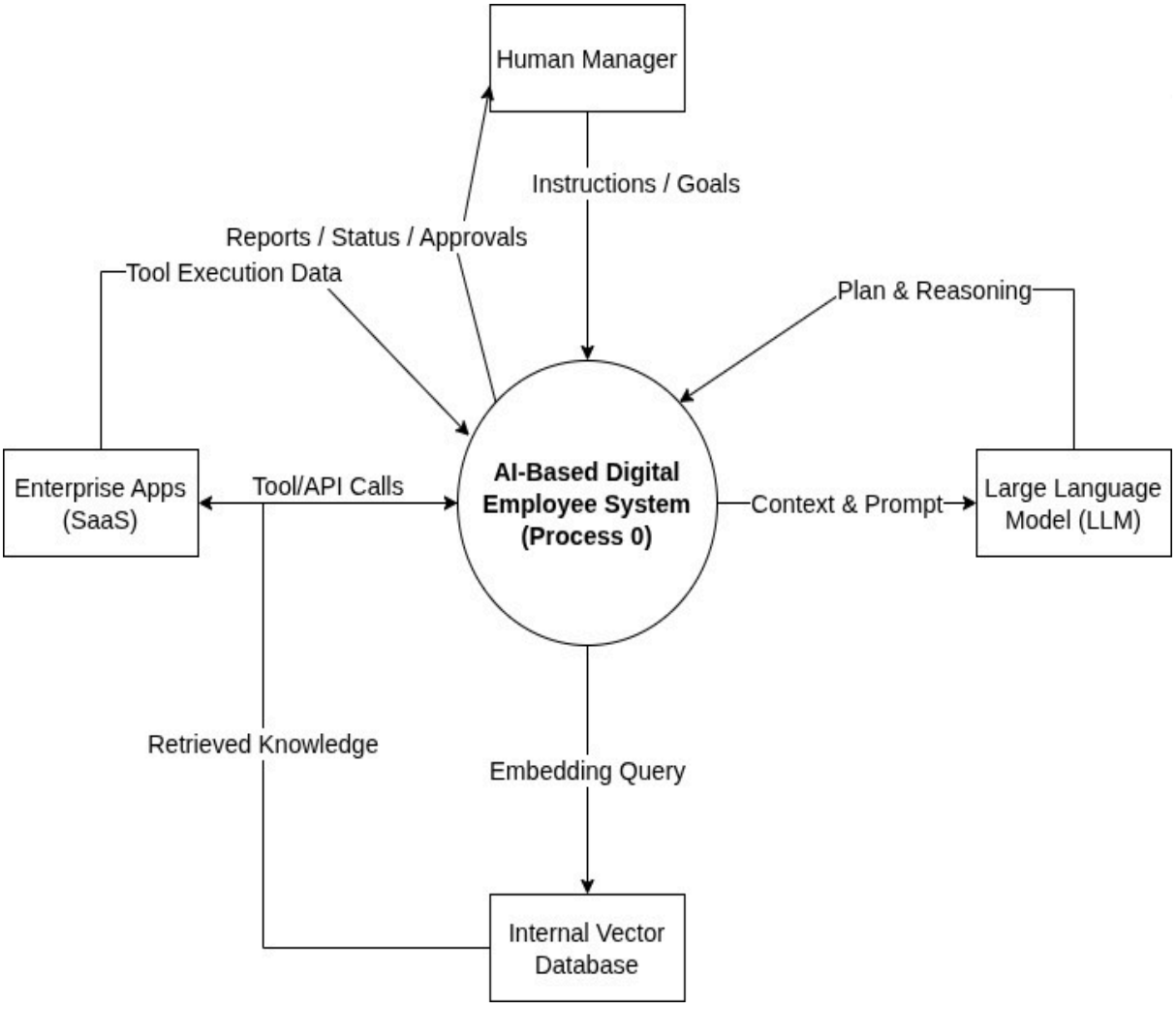

Figure 2

4.2. DFD Level 1

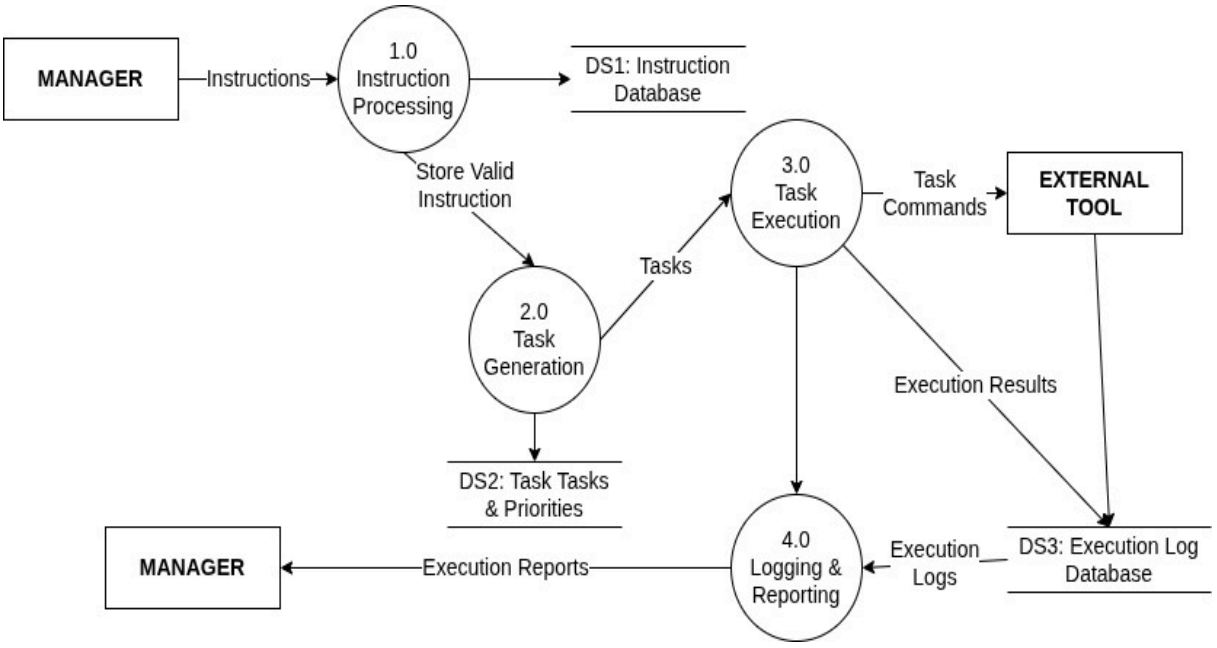

Figure 3

4.3. DFD Level 2 (Detailed Breakdown)

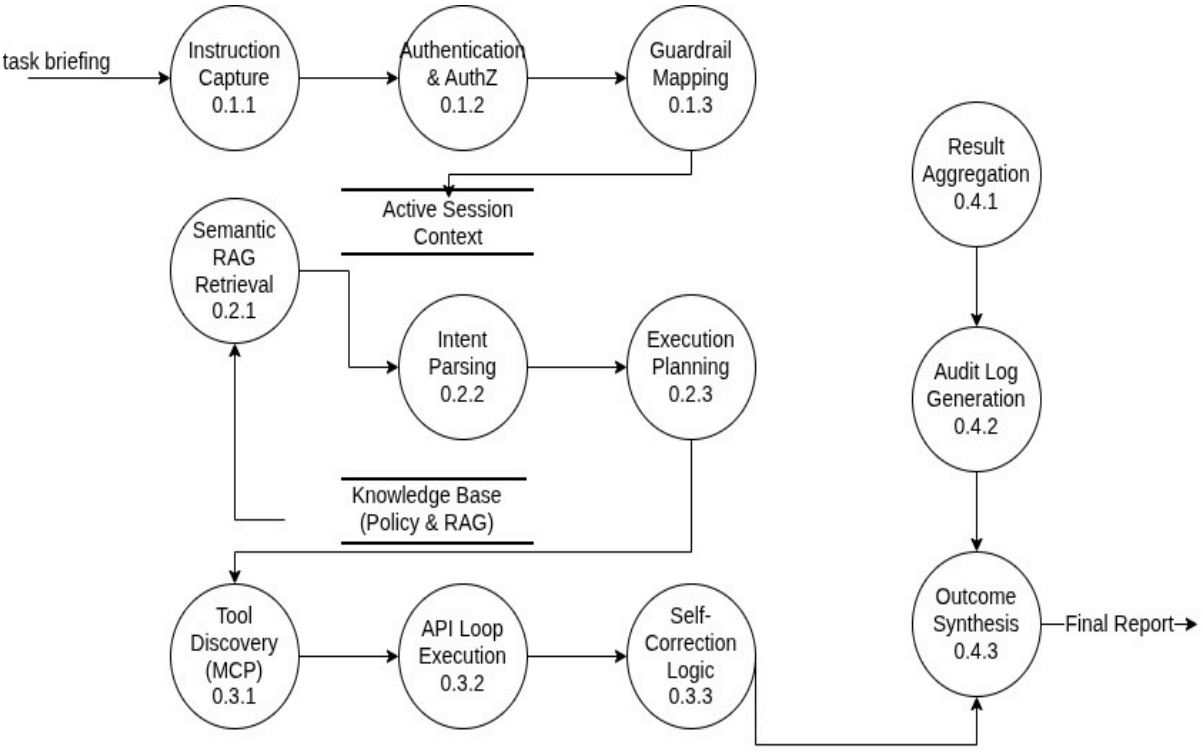

Figure 4

4.4 STRUCTURE CHART

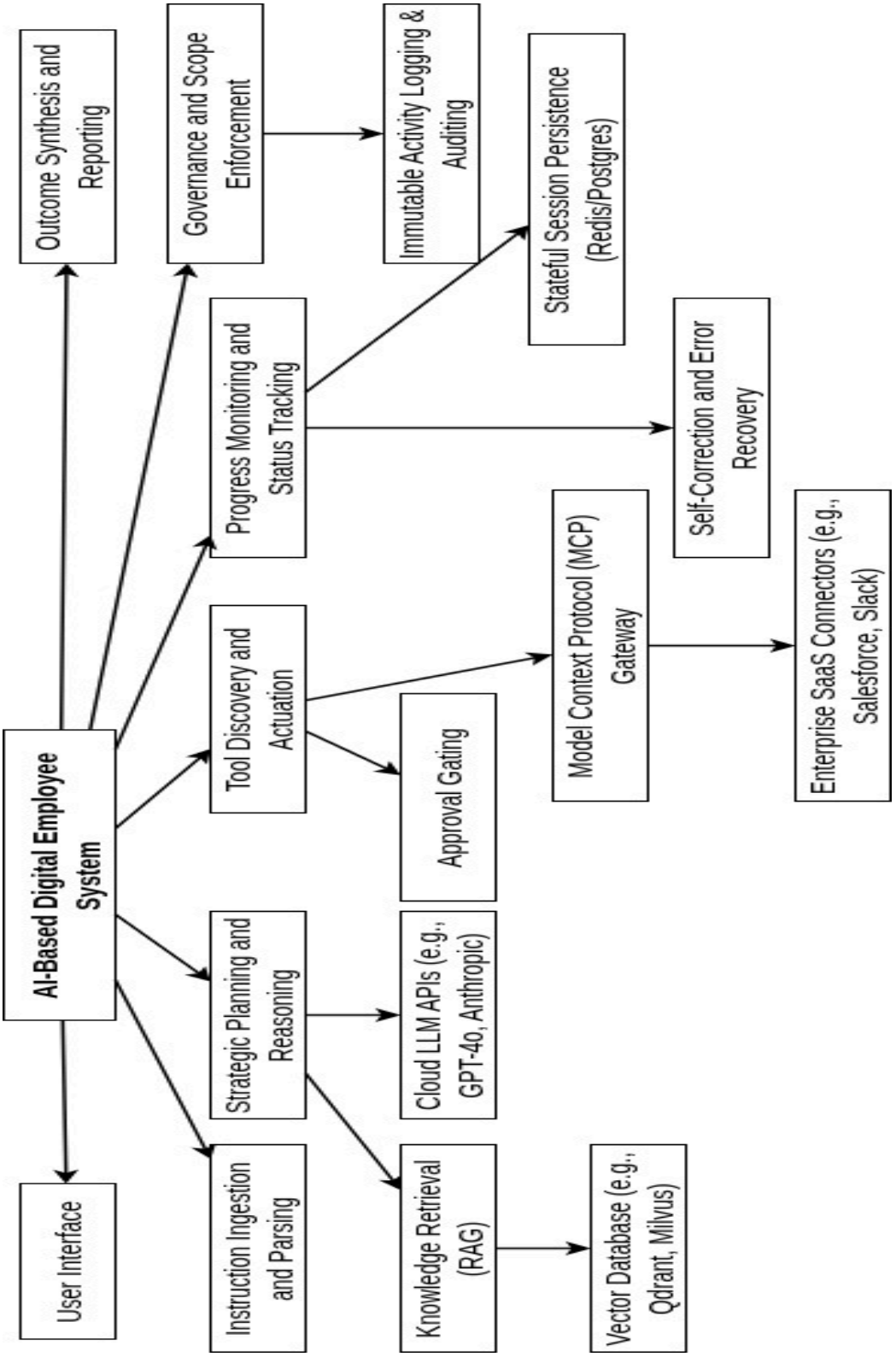

Figure 5

5. UML Diagrams

5.1 Use Case Diagram and User Flow

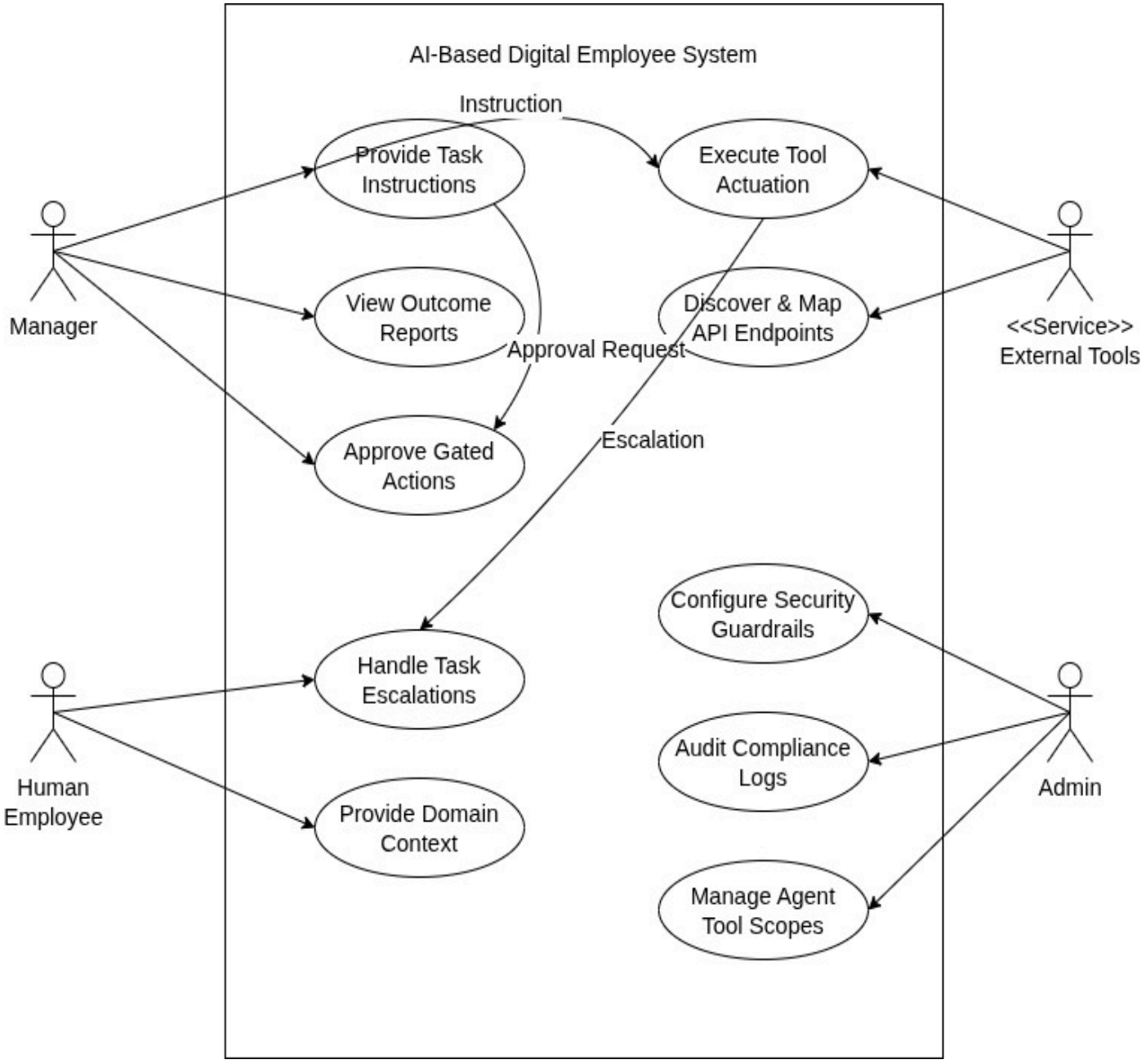

Figure 6

5.2 CLASS DIAGRAM

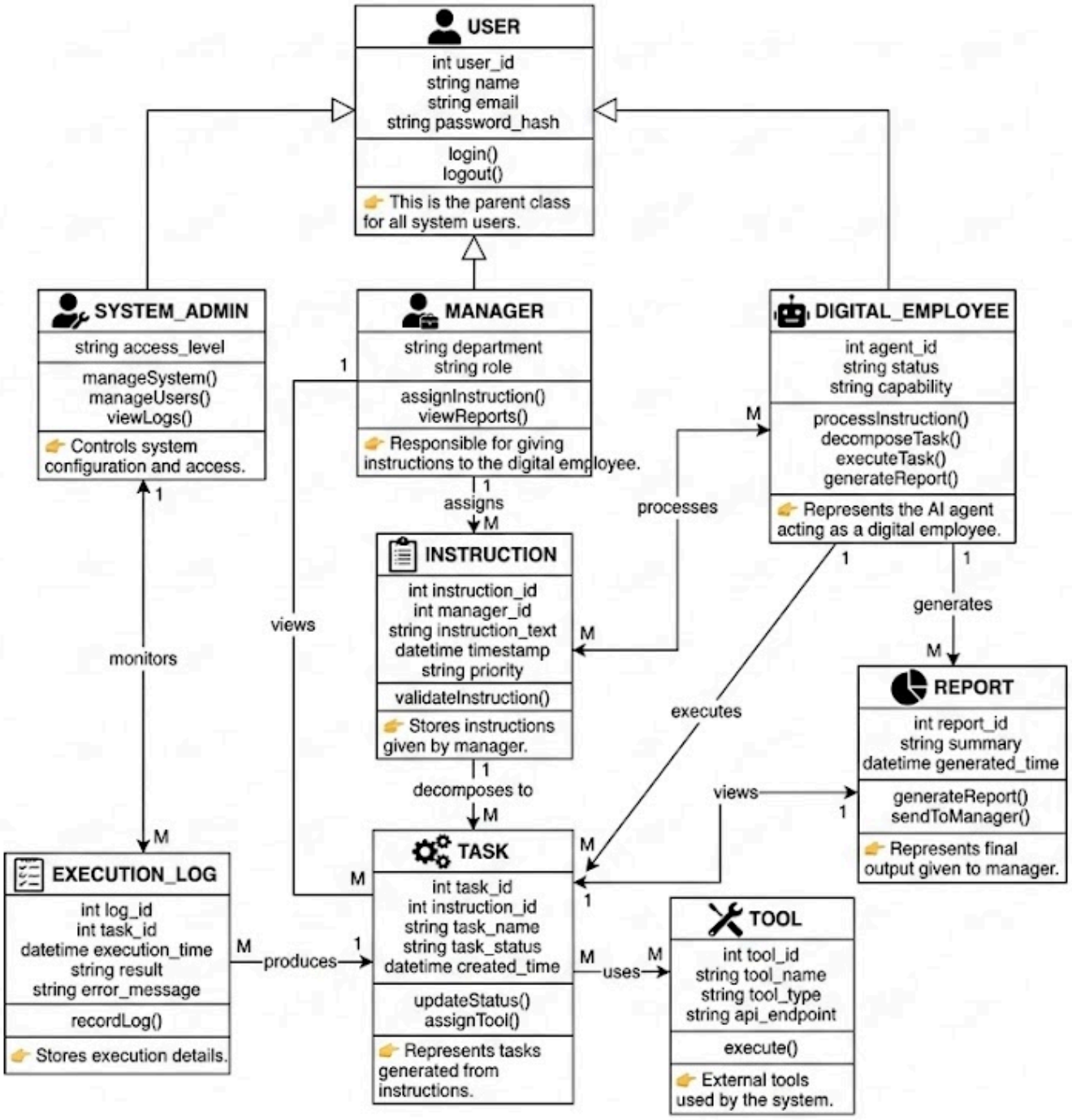

Figure 7

5.3 OBJECT DIAGRAM

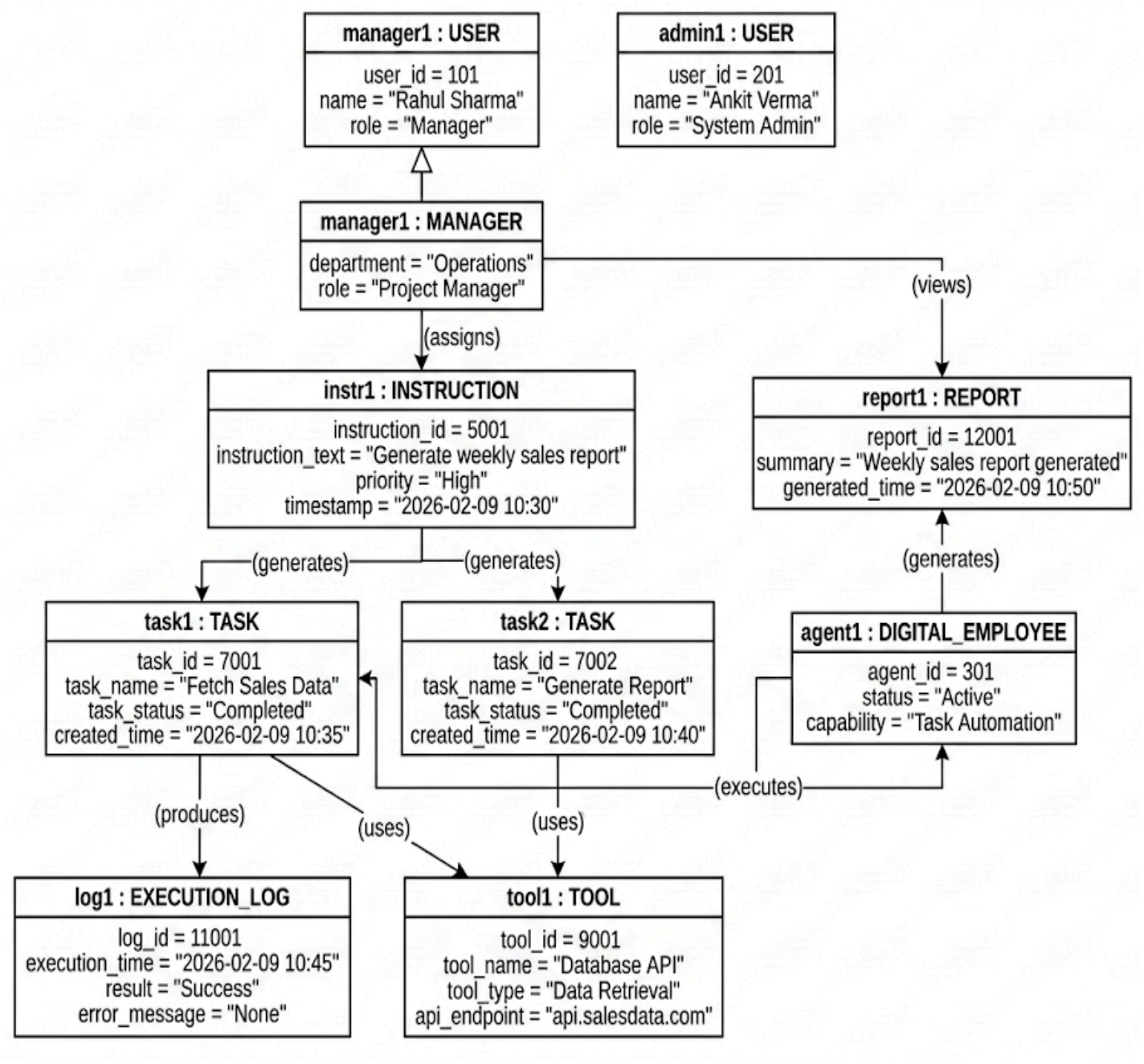

Figure 8

5.4. Activity Diagram (User Flow)

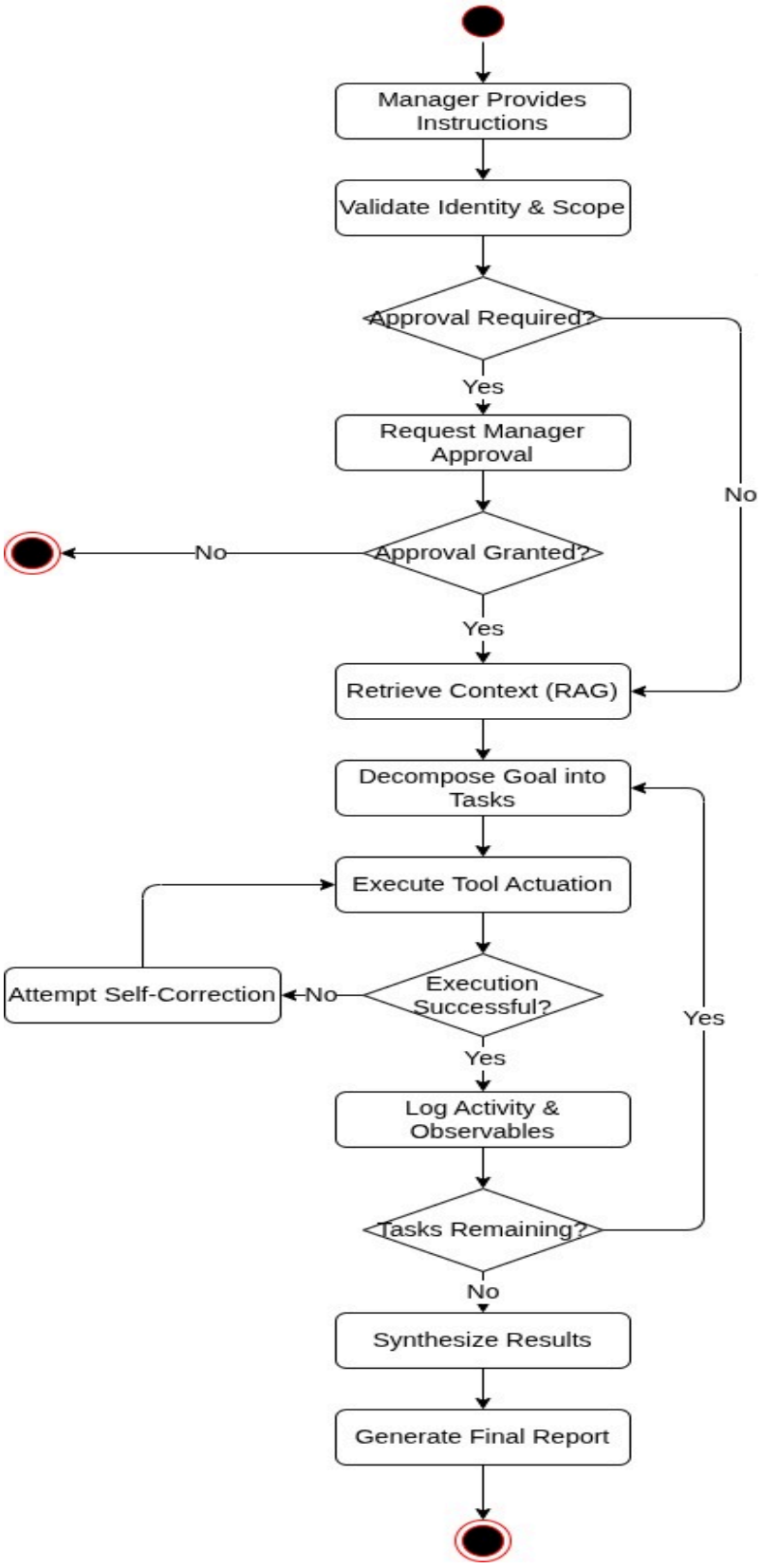

Figure 9

5.5 STATE CHART DIAGRAM

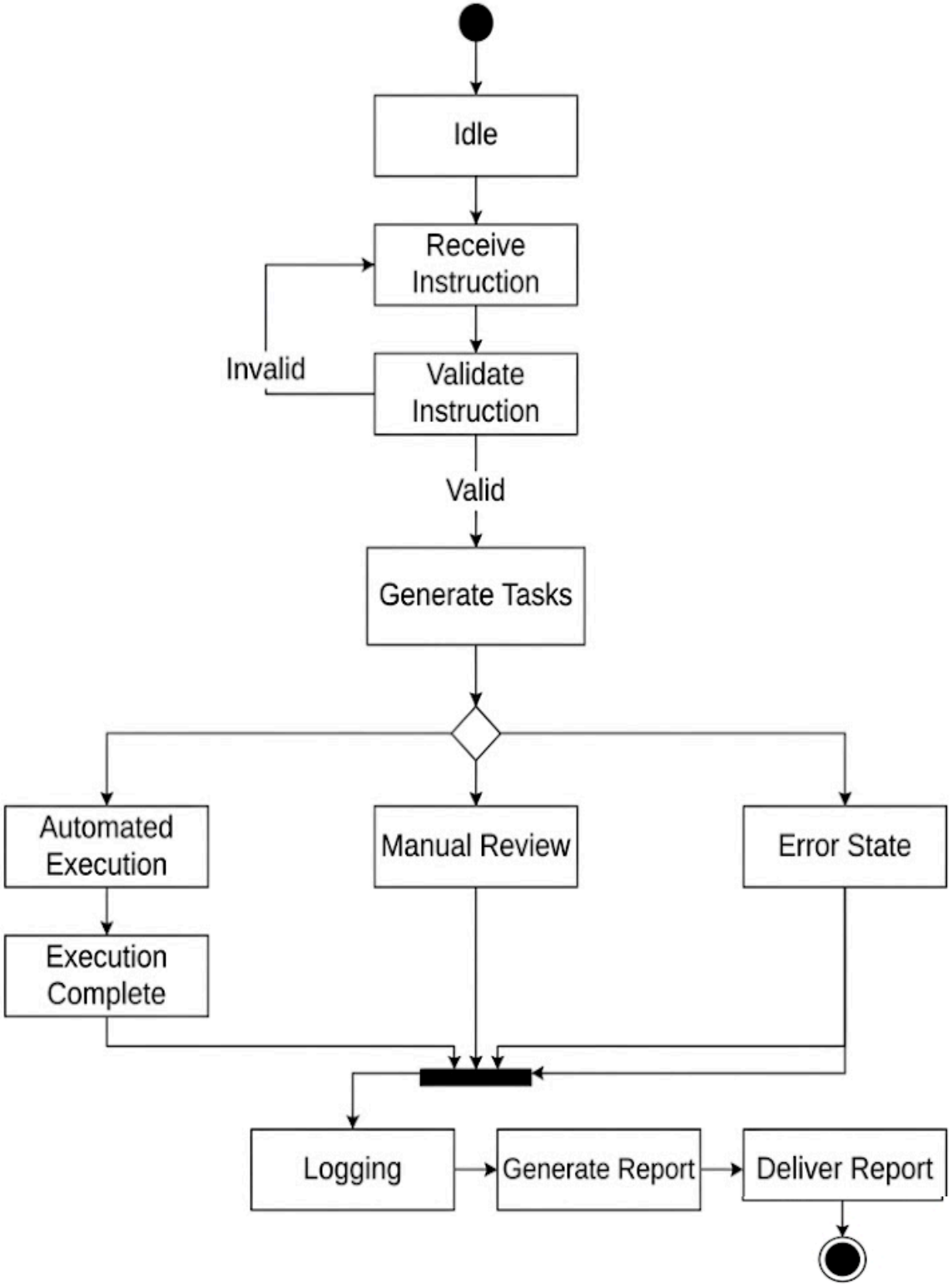

Figure 10

5.6. Sequence Diagram

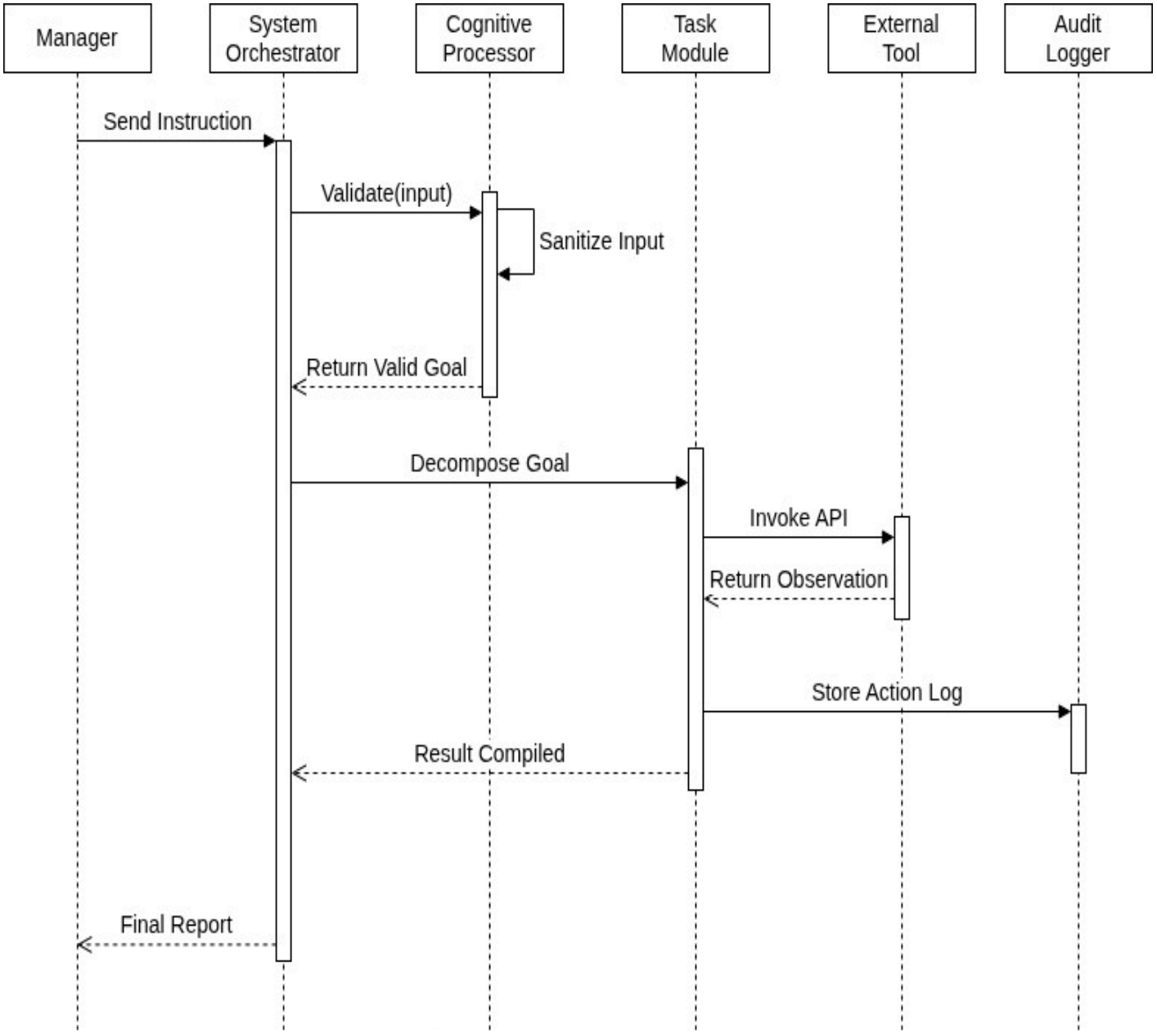

Figure 11

5.7 COLLABORATION DIAGRAM

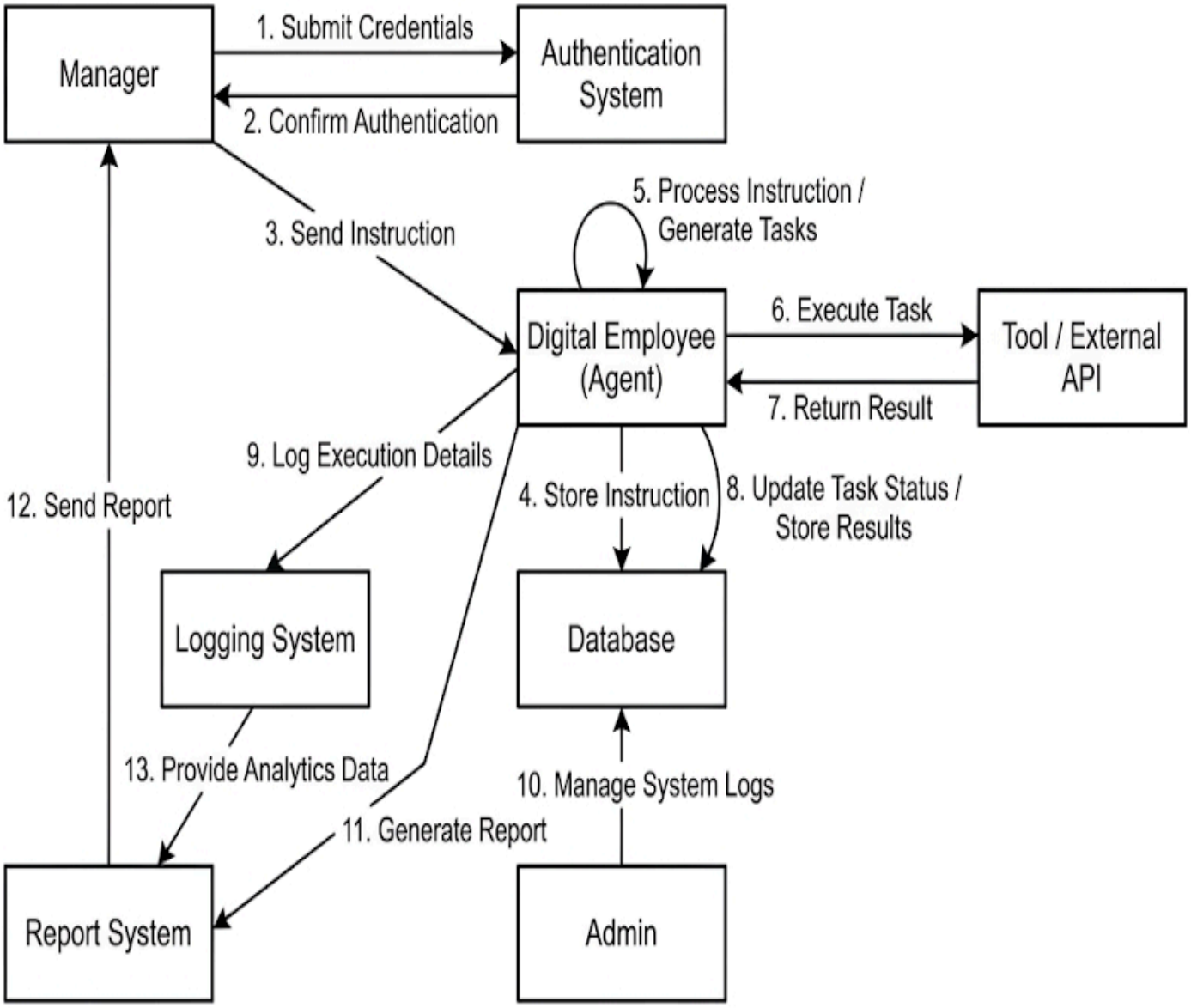

Figure 12

6. Conclusion

The AI-Based Digital Employee Using an Agent system was successfully analyzed and modeled through various software engineering techniques. The project began with the identification of the problem and key stakeholders, followed by requirement analysis to define both functional and non-functional requirements of the system.

A structured Software Requirement Specification (SRS) was developed to clearly document system behavior and constraints. The Entity Relationship (ER) model provided a clear representation of data entities, attributes, and relationships, forming the foundation for database design. Data Flow Diagrams (DFDs) illustrate how data moves through different levels of the system, from instruction input to task execution and reporting.

Further, UML diagrams such as Use Case, Activity, and Sequence diagrams were used to represent user interactions, system workflows, and internal communication between components. These models collectively provide a comprehensive understanding of how the digital employee operates, processes instructions, executes tasks, and generates outputs efficiently.

Overall, the project demonstrates how an AI-driven agent can function as a digital employee by automating routine tasks while maintaining control, transparency, and scalability. The use of structured modeling techniques ensures that the system design is clear, organized, and ready for implementation in real-world scenarios.

Samuel Priyatam Dash
Regd No. - 2302040025